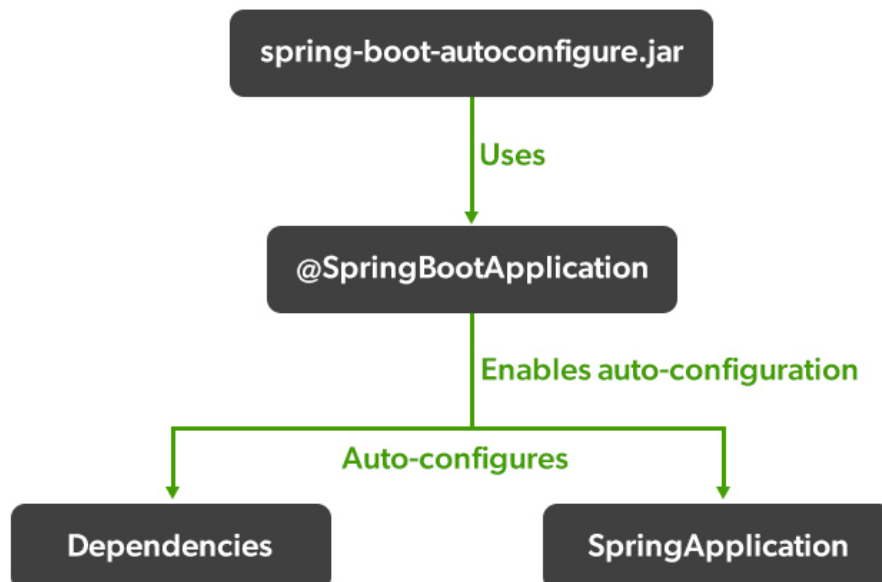


SpringBoot

Spring Boot — Auto-Configuration

What is Auto-Configuration?

- Auto-Configuration is a key feature of Spring Boot.
- It automatically configures Spring components based on:
 1. Dependencies present in the classpath
 2. Existing beans
 3. Application properties
- Removes the need for manual configuration in XML or Java.



Auto-Configuration Annotations

A) @SpringBootApplication

This annotation contains:

1. `@Configuration` → Marks class as configuration
2. `@EnableAutoConfiguration` → Enables auto-config
3. `@ComponentScan` → Scans components automatically

B) @EnableAutoConfiguration (internal)

- Triggers auto-configuration mechanism.
- Rarely used directly, because `@SpringBootApplication` already uses it.

How Auto-Configuration Works? (Internal Process)

When Spring Boot starts:

1. It checks dependencies on the classpath
e.g., spring-webmvc, spring-data-jpa, spring-security
2. Based on these, it loads default configurations.
3. It reads application.properties or application.yml to apply custom values.
4. It creates required beans automatically (DataSource, DispatcherServlet, EntityManagerFactory, etc.).

Examples of What Auto-Configuration Provides

A) Spring Web (spring-webmvc)

- Auto creates DispatcherServlet
- Auto-configures MVC

B) Spring Data JPA

- Auto creates DataSource
- Sets Hibernate properties

C) Hibernate / JPA

- Configures transaction manager
- Maps entities

D) Embedded Server

- Configures Tomcat/Jetty/Undertow
- Sets default port 8080

E) Static Resources

- Serves static files automatically from:
 1. /static/
 2. /public/
 3. /resources/

F) Spring Security

- Autoconfigures basic login form when dependency exists

Spring Boot – Application Properties

What is application.properties?

- It is the main configuration file in Spring Boot projects.
- Located in:
src/main/resources/application.properties.
- Used to customize application behavior without touching the Java code.
- Spring Boot reads this file automatically during startup.

Why use Application Properties?

- To override default Spring Boot settings.
- To configure server, DB connection, logging, security, mail, etc.
- Keeps code clean and configuration centralized.

Common Uses of application.properties

A) Server Configuration

- Change server port:
- server.port=9090
- Change application context path:
- server.servlet.context-path=/myapp

B) Spring Application Name

- spring.application.name=My_SpringBoot_App
- Used in logs and Actuator endpoints.

C) Database Configuration (Example: PostgreSQL)

- spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
- spring.datasource.username=postgres
- spring.datasource.password=1234
- spring.jpa.hibernate.ddl-auto=update
- spring.jpa.show-sql=true

D) Logging Configuration

- Set logging level:
- `logging.level.org.springframework=INFO`
- `logging.level.com.project=DEBUG`
- Change log file:
- `logging.file.name=app.log`

E) Static Resources Configuration

(Only if customizing default)

`spring.web.resources.static-locations=classpath:/assets/,classpath:/public/`

F) Thymeleaf / JSP View Configuration

(For JSP)

`spring.mvc.view.prefix=/WEB-INF/views/`

`spring.mvc.view.suffix=.jsp`

Spring Boot – Spring Data JPA

What is Spring Data JPA?

- A part of Spring framework used to interact with databases.
- Built on top of **JPA (Java Persistence API)**.
- Simplifies database operations using **Repository interfaces**.
- Removes boilerplate code like writing SQL queries manually.

Why Use Spring Data JPA?

- Automatic query creation.
- Reduces code significantly (CRUD without writing SQL).
- Works with different databases (MySQL, PostgreSQL, Oracle, etc.).
- Provides pagination, sorting, and custom queries.
- Integrates easily with Spring Boot.

Key Components of Spring Data JPA

A) Entity

- Represents a table in the database.
 - Annotated with `@Entity`.
 - Each field → table column.
- Example:

`@Entity`

```
public class Student {  
  
    @Id  
    private int id;  
    private String name;  
}
```

B) Repository

- Interface that interacts with the DB.

- You do NOT write implementation; Spring Boot auto-generates it.

Types:

1. CrudRepository – Basic CRUD
2. JpaRepository – CRUD + pagination + sorting
3. PagingAndSortingRepository – Sorting/Paging

Example:

```
public interface StudentRepository extends JpaRepository<Student, Integer> {}
```

C) Service Layer

- Business logic layer.
- Calls repository methods.

Example:

```
@Service
```

```
public class StudentService {
```

```
    @Autowired
```

```
    StudentRepository repo;
```

```
}
```

D) JPA Provider

- Default provider → **Hibernate**.
- Converts Java objects to SQL queries and vice versa.

Common Annotations in JPA

Entity-Level

1. @Entity → marks as DB table
2. @Table(name="table_name") → optional custom table name

Field-Level

1. @Id → primary key
2. @GeneratedValue → auto-increment

3. `@Column(name="col_name")` → custom column

Relationships

1. `@OneToOne`
2. `@OneToMany`
3. `@ManyToOne`
4. `@ManyToMany`
5. `@JoinColumn`

CRUD Operations (No SQL Needed)

Spring Data JPA gives ready-made methods:

1. **Save** - `repo.save(entity);`
2. **Find All** - `repo.findAll();`
3. **Find by Id** - `repo.findById(10);`
4. **Delete** - `repo.deleteById(10);`

Spring Data JPA – CRUD Repository

What is CrudRepository?

- CrudRepository is an interface provided by Spring Data JPA.
- Used to perform Basic CRUD (Create, Read, Update, Delete) operations on a database.
- It reduces boilerplate code — No need to write SQL or DAO implementation.
- Spring automatically creates the implementation at runtime.

CRUD Repository Signature

- `public interface CrudRepository<T, ID>`

Where:

- `T` → Entity class (e.g., Student, Employee)
- `ID` → Type of primary key (e.g., Integer, Long)

Example:

```
public interface StudentRepository extends CrudRepository<Student, Integer> { }
```

Features of CrudRepository

- Provides ready-to-use CRUD methods.
- No custom implementation required.
- Supports generic types.
- Integrated with Spring Boot and Hibernate.
- Good for small projects; for advanced operations use JpaRepository.

Important Methods in CrudRepository

A) Create / Update

```
<S extends T> S save(S entity);
```

- Inserts if ID is new
- Updates if ID already exists

B) Read (Retrieve Records)

Find All - `Iterable<T> findAll();`

Find by ID - `Optional<T> findById(ID id);`

Count Records - `long count();`

C) Delete

Delete by ID

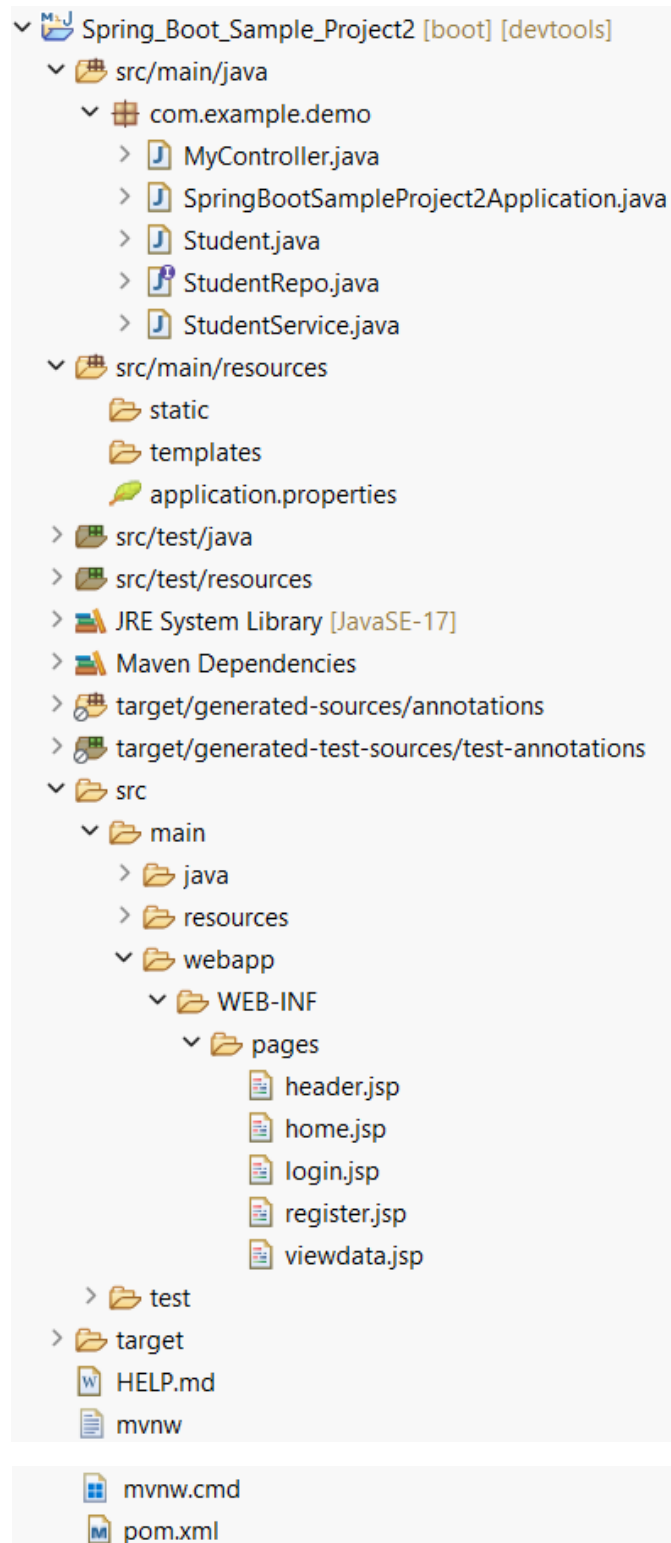
`void deleteById(ID id)` - Delete All Records

`void deleteAll()` - Delete Specific Entity

`void delete(T entity);`

Example Project

Project Structure -



pom.xml –

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>Spring_Boot_Sample_Project</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>Spring_Boot_Sample_Project</name>
  <description>Demo project for Spring Boot</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies> Add Spring Boot Starters...
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.apache.tomcat.embed/tomcat-embed-jasper -->
    <dependency>
      <groupId>org.apache.tomcat.embed</groupId>
      <artifactId>tomcat-embed-jasper</artifactId>
      <scope>provided</scope>
    </dependency>

    <!-- jsp pages -->
    <!-- https://mvnrepository.com/artifact/jakarta.servlet.jsp.jstl/jakarta.servlet.jsp.jstl-api -->
    <dependency>
      <groupId>jakarta.servlet.jsp.jstl</groupId>
      <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
    </dependency>

    <dependency>
      <groupId>org.glassfish.web</groupId>
      <artifactId>jakarta.servlet.jsp.jstl</artifactId>
    </dependency>

    <!-- Database -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Student.java –

```
package com.example.demo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String fullname;

    @Column(unique=true)
    private String email;

    @Column
    private String password;

    @Column
    private String address;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFullname() {
        return fullname;
    }

    public void setFullname(String fullname) {
        this.fullname = fullname;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", fullname=" + fullname + ", email=" + email + ", password=" + password
            + ", address=" + address + "]";
    }
}
```

StudentRepo.java –

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepo extends JpaRepository<Student, Long> {

}
```

MyController.java –

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class MyController {

    @Autowired
    StudentService ss;

    @GetMapping("/")
    public String home()
    {
        return "home";    //home.jsp
    }

    @GetMapping("/register")
    public String register()
    {
        return "register";    //register.jsp
    }

    @GetMapping("/viewdata")
    public String viewdata()
    {
        return "viewdata";    //viewdata.jsp
    }

    @GetMapping("/login")
    public String login()
    {
        return "login";    //login.jsp
    }

    @PostMapping("/saveform")
    public void saveform(@ModelAttribute Student s) {
        ss.registration(s);
    }
}
```

StudentService.java –

```
package com.example.demo;

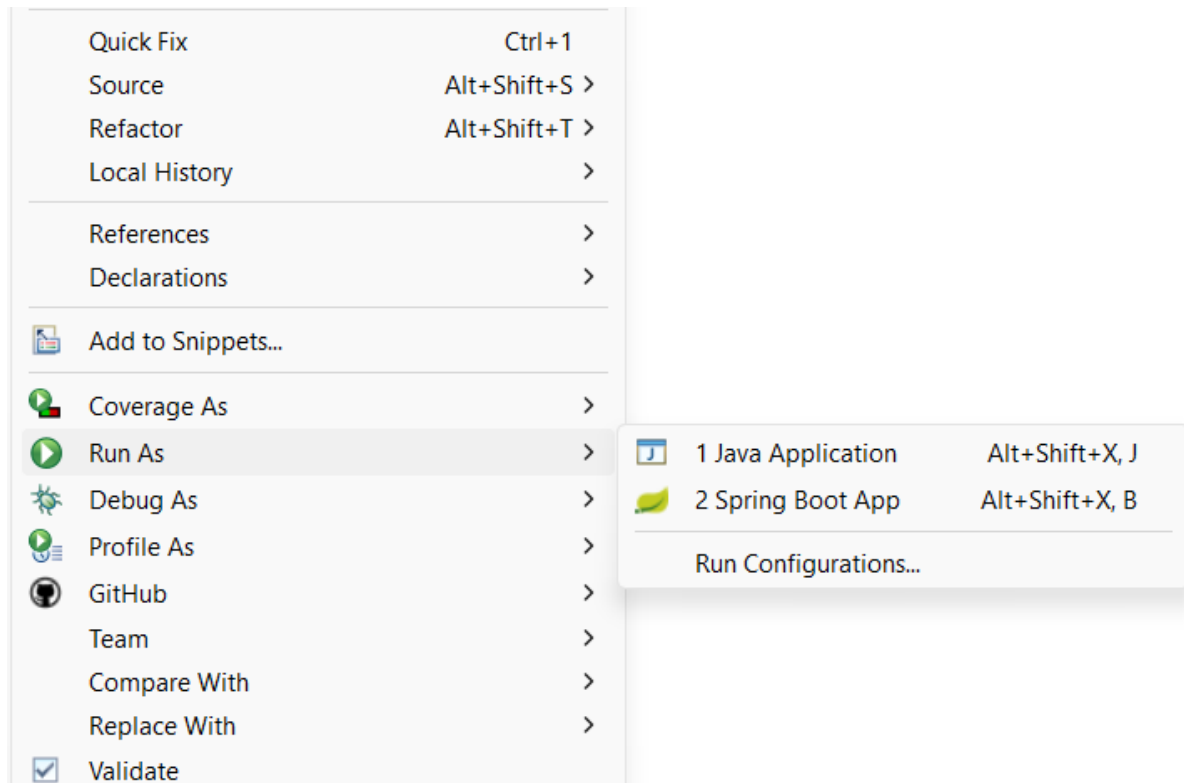
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class StudentService {

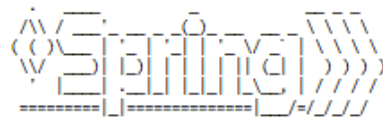
    @Autowired
    StudentRepo sr;

    public void registration(Student s)
    {
        sr.save(s);    //Insert
    }
}
```

When everything is correct, easily this can be run as a normal Java application.



Console Output –



```
:: Spring Boot :: (v4.0.0)

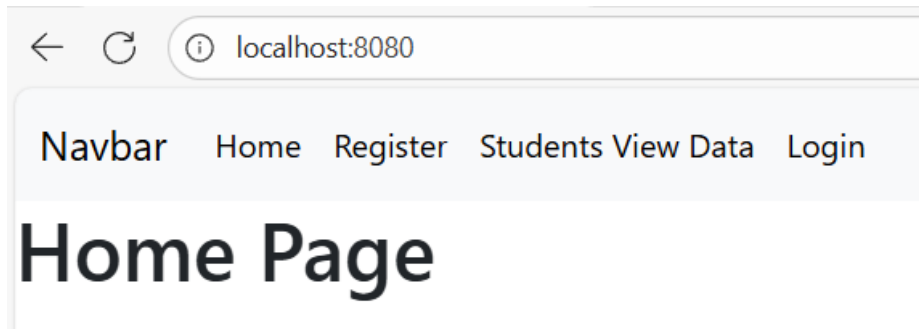
2025-11-28T18:48:28.788+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.d.SpringBootSampleProject2Application
/Users\sagar\eclipse-workspace\Spring_Boot_Sample_Project2
2025-11-28T18:48:28.795+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.d.SpringBootSampleProject2Application
2025-11-28T18:48:28.898+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2025-11-28T18:48:28.898+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2025-11-28T18:48:30.480+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate
2025-11-28T18:48:30.581+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate
2025-11-28T18:48:31.469+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.s.boot.tomcat.TomcatWebServer
2025-11-28T18:48:31.496+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.apache.catalina.core.StandardService
2025-11-28T18:48:31.496+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.apache.catalina.core.StandardEngine
2025-11-28T18:48:31.896+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] org.apache.jasper.servlet.JdScanner
Skipping unneeded JARs during scanning can improve startup time and JSP compilation time.
2025-11-28T18:48:31.918+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] b.w.c.s.WebApplicationContextInitializer
2025-11-28T18:48:32.197+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper
2025-11-28T18:48:32.308+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] org.hibernate.Version
2025-11-28T18:48:33.319+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo
2025-11-28T18:48:33.396+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] com.zaxxer.hikari.HikariDataSource
2025-11-28T18:48:33.740+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] com.zaxxer.hikari.pool.HikariPool
2025-11-28T18:48:33.743+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] com.zaxxer.hikari.HikariDataSource
2025-11-28T18:48:33.901+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] org.hibernate.orm.connections.pooling

Database JDBC URL [jdbc:postgresql://localhost:5433/springbootdb]
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 18.0
Default catalog/schema: springbootdb/public
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DataSourceConnectionProviderImpl
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-11-28T18:48:35.834+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator
2025-11-28T18:48:35.966+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] j.LocalContainerEntityManagerFactoryBean
2025-11-28T18:48:36.138+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.s.d.j.r.query.QueryEnhancerFactories
2025-11-28T18:48:36.363+05:30 WARN 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration
warning
2025-11-28T18:48:37.012+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.s.boot.tomcat.TomcatWebServer
2025-11-28T18:48:37.022+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.d.SpringBootSampleProject2Application
2025-11-28T18:49:17.386+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]
2025-11-28T18:49:17.386+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2025-11-28T18:49:17.391+05:30 INFO 31944 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2025-11-28T18:49:19.310+05:30 WARN 31944 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-2] o.s.web.servlet.PageNotFound
2025-11-28T18:49:19.325+05:30 WARN 31944 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-2] o.s.web.servlet.PageNotFound

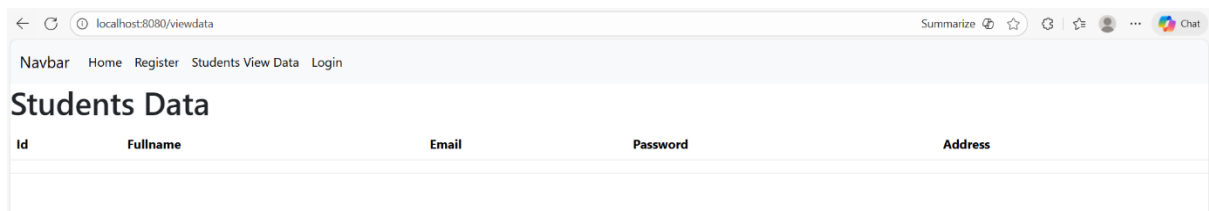
Hibernate:
insert
into
student
(address, email, fullname, password)
values
(?, ?, ?, ?)
```

Web Based –

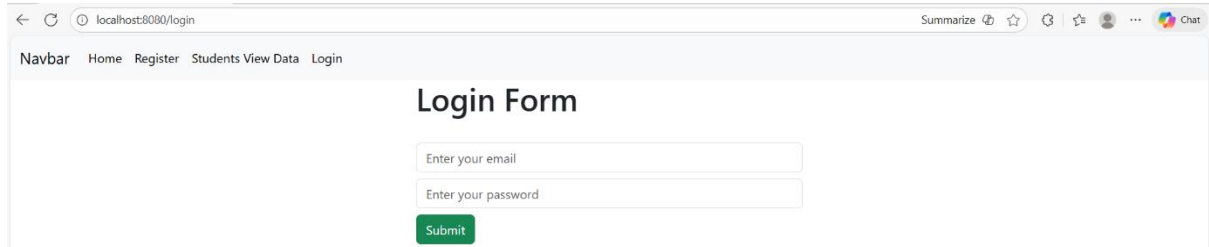
Home Page -



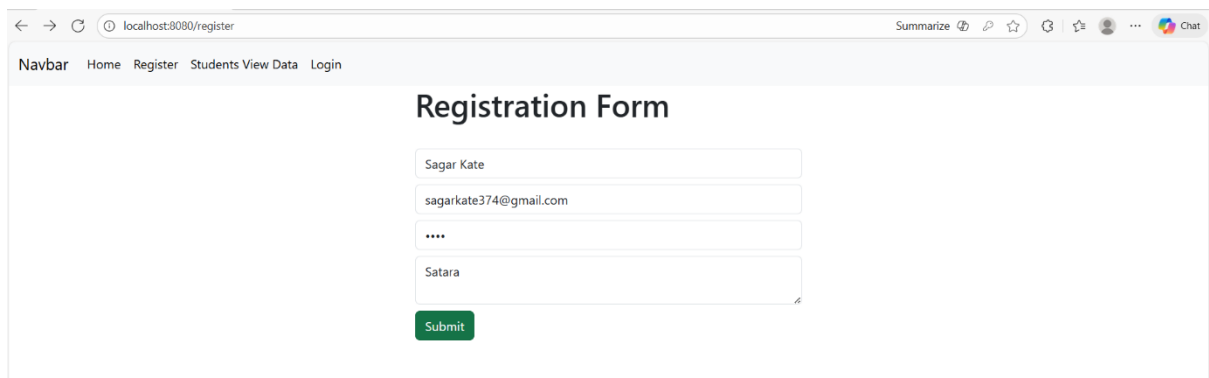
Viewdata Page –



Login Page –



Register Page –



After register fill the data & submit form then –



Show the Database –

	id [PK] bigint	address character varying (255)	email character varying (255)	fullname character varying (255)	password character varying (255)
1	1	Satara	sagarkate374@gmail.c...	Sagar Kate	8520